AI-ASSISTED CODING

Roll Number:2403A51457

AI-End LAb Exam

SET-6

Q1. AI Code Completion (Scenario)

A finance company needs an `AccountLedger` class to compute:

- deposits
- withdrawals
- running balance

Use AI-assisted completion to finish the class.

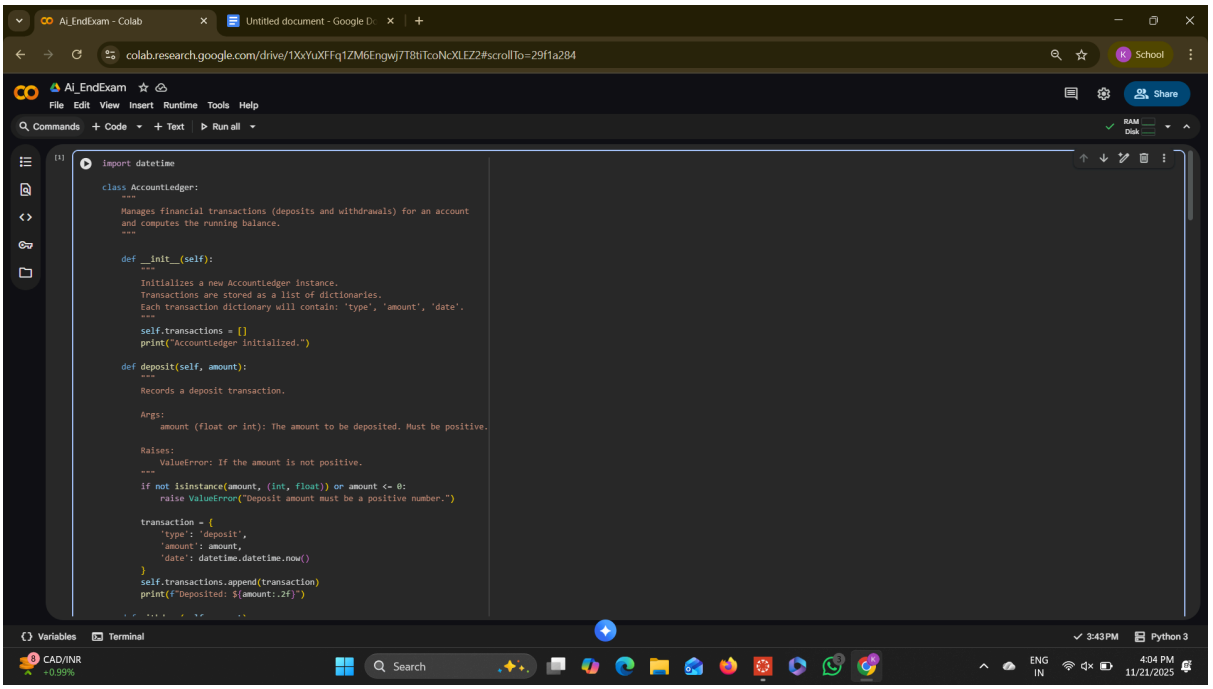
(Include documentation + comments + test cases.)

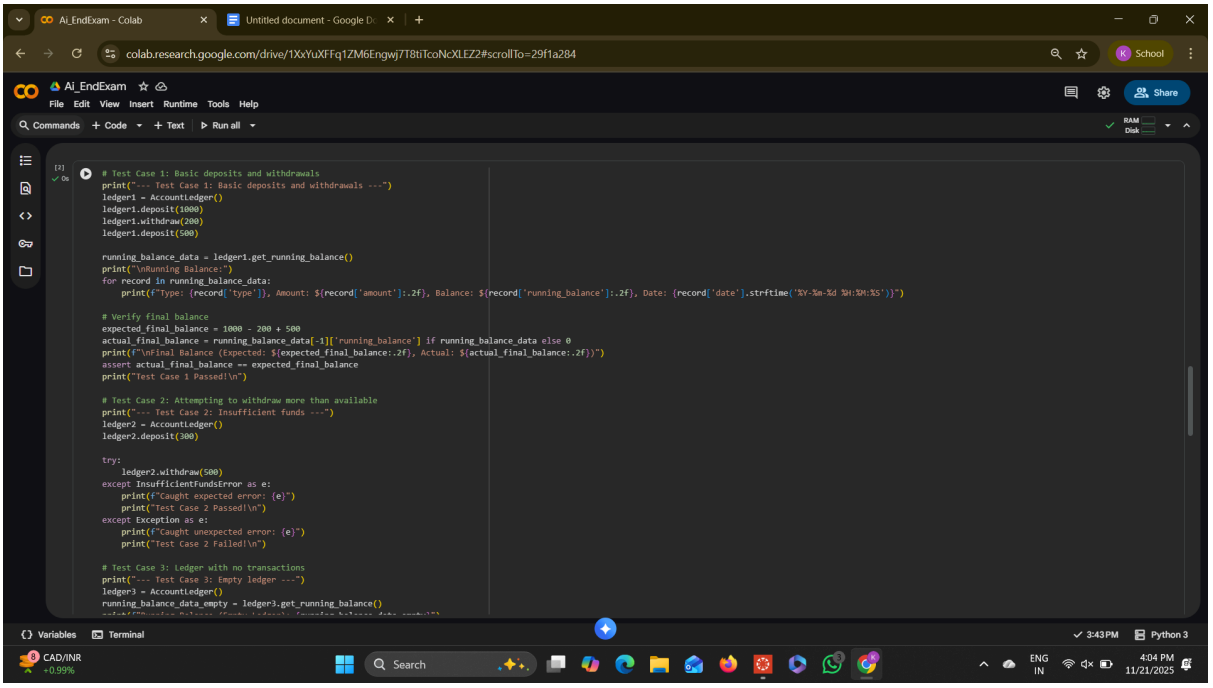
PROMPT:

Create a Finance Company needs an AccountLedger class to compute:

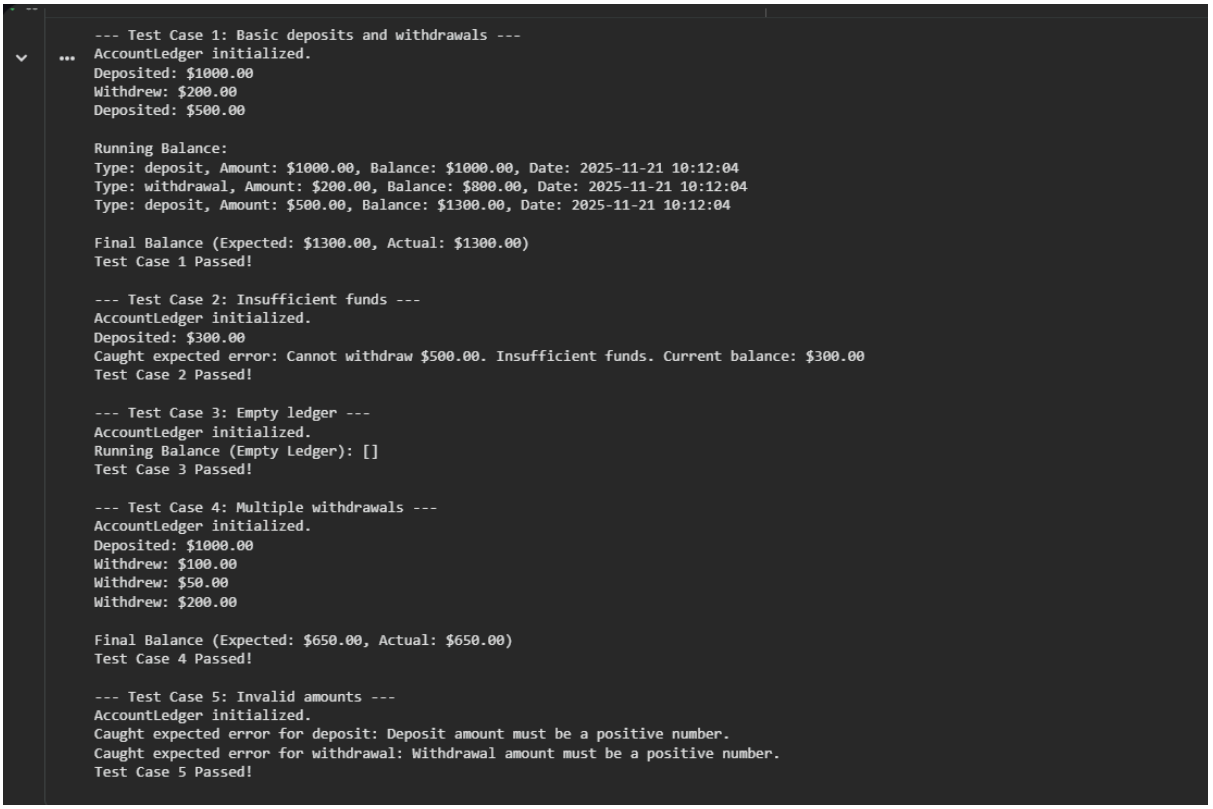
- deposits
- withdrawals
- running balance

INPUT:


Ai_EndExam - Colab
Untitled document - Google D
colab.research.google.com/drive/1XxYuXFFq1ZM6Engwj7T8tiTcoNcXLEZ2#scrollTo=29f1a284
Ai_EndExam
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
import datetime
class AccountLedger:
"""
Manages financial transactions (deposits and withdrawals) for an account
and computes the running balance.
"""
def __init__(self):
"""
Initializes a new AccountLedger instance.
Transactions are stored as a list of dictionaries.
Each transaction dictionary will contain: 'type', 'amount', 'date'.
"""
self.transactions = []
print("AccountLedger initialized.")
def deposit(self, amount):
"""
Records a deposit transaction.
Args:
amount (float or int): The amount to be deposited. Must be positive.
Raises:
ValueError: If the amount is not positive.
"""
if not isinstance(amount, (int, float)) or amount <= 0:
raise ValueError("Deposit amount must be a positive number.")
transaction = {
'type': 'deposit',
'amount': amount,
'date': datetime.datetime.now()
}
self.transactions.append(transaction)
print(f"Deposited: ${amount:.2f}")
Variables
Terminal
3:43 PM
Python 3
CAD/INR
+0.99%
Search
ENG
IN
4:04 PM
11/21/2025


Ai_EndExam - Colab
Untitled document - Google D
colab.research.google.com/drive/1XxYuXFFq1ZM6Engwj7T8tiTcoNcXLEZ2#scrollTo=29f1a284
Ai_EndExam
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
# Test Case 1: Basic deposits and withdrawals
print("--- Test Case 1: Basic deposits and withdrawals ---")
ledger1 = AccountLedger()
ledger1.deposit(1000)
ledger1.withdraw(200)
ledger1.deposit(500)
running_balance_data = ledger1.get_running_balance()
print("\nRunning Balance:")
for record in running_balance_data:
print(f"Type: {record['type']}, Amount: ${record['amount']:.2f}, Balance: ${record['running_balance']:.2f}, Date: {record['date'].strftime('%Y-%m-%d %H:%M:%S')}")
# Verify final balance
expected_final_balance = 1000 - 200 + 500
actual_final_balance = running_balance_data[-1]['running_balance'] if running_balance_data else 0
print(f"\nFinal Balance (Expected: ${expected_final_balance:.2f}, Actual: ${actual_final_balance:.2f})")
assert actual_final_balance == expected_final_balance
print("Test Case 1 Passed!\n")
# Test Case 2: Attempting to withdraw more than available
print("--- Test Case 2: Insufficient funds ---")
ledger2 = AccountLedger()
ledger2.deposit(300)
try:
ledger2.withdraw(500)
except InsufficientFundsError as e:
print(f"Caught expected error: {e}")
print("Test Case 2 Passed!\n")
except Exception as e:
print(f"Caught unexpected error: {e}")
print("Test Case 2 Failed!\n")
# Test Case 3: Ledger with no transactions
print("--- Test Case 3: Empty ledger ---")
ledger3 = AccountLedger()
running_balance_data_empty = ledger3.get_running_balance()
Variables
Terminal
3:43 PM
Python 3
CAD/INR
+0.99%
Search
ENG
IN
4:04 PM
11/21/2025

OUTPUT:



Q2. Algorithms (Scenario)

A document search tool needs both
Sequential Search
and
Fibonacci Search

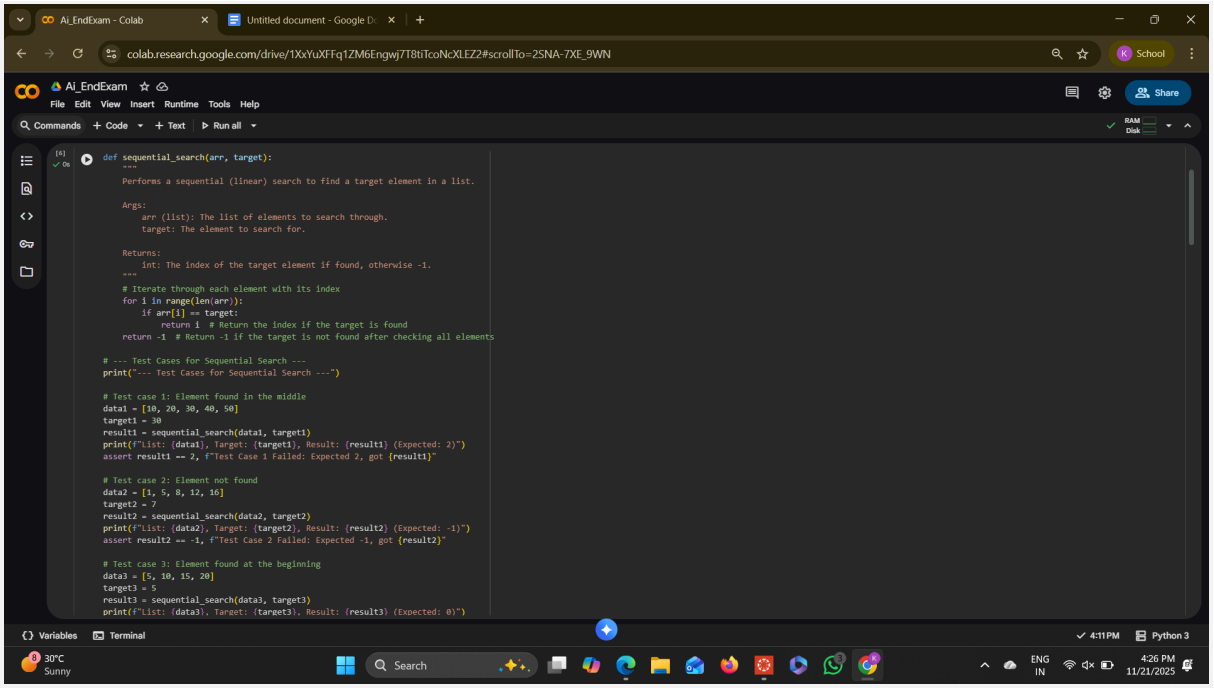
implementations.

(Include documentation + examples + tests.)

PROMPT:
Create a Document search tool needs both Sequential Search and Fibonacci Search Implementations

INPUT:

For Sequential Search:

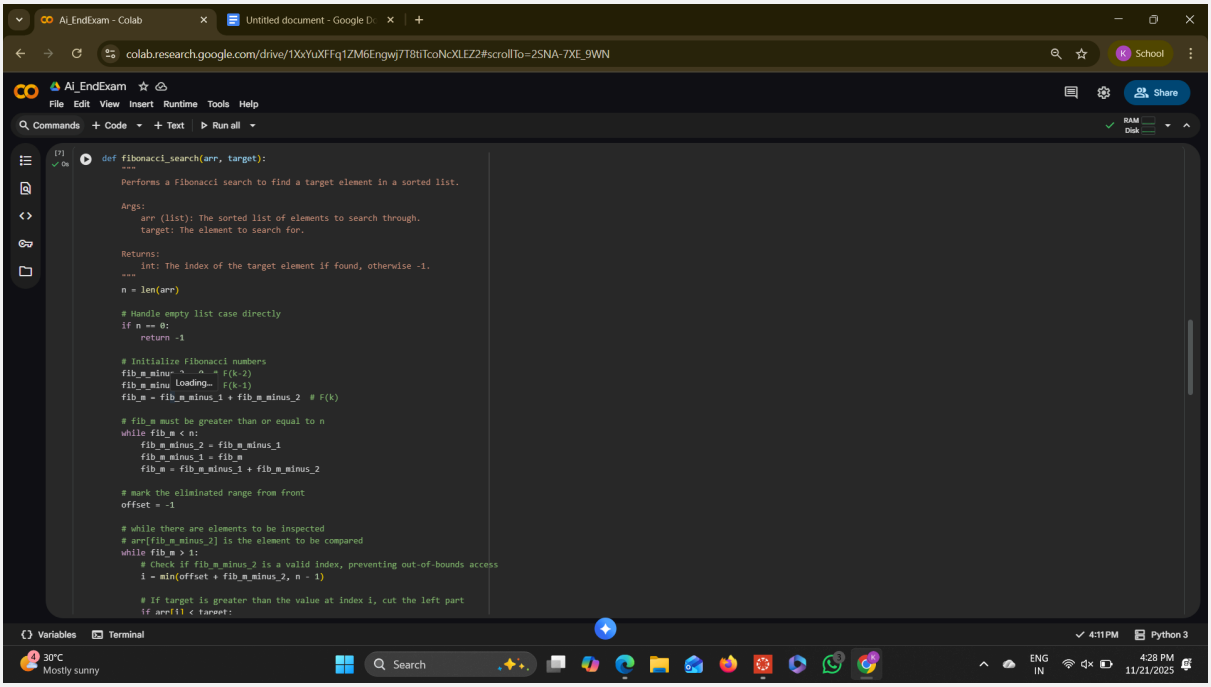


OUTPUT:

```
--- Test Cases for Sequential Search ---
List: [10, 20, 30, 40, 50], Target: 30, Result: 2 (Expected: 2)
List: [1, 5, 8, 12, 16], Target: 7, Result: -1 (Expected: -1)
List: [5, 10, 15, 20], Target: 5, Result: 0 (Expected: 0)
List: [100, 200, 300, 400], Target: 400, Result: 3 (Expected: 3)
List: [], Target: 10, Result: -1 (Expected: -1)
All Sequential Search test cases passed!
```

INPUT:

For Fibonacci Search:



OUTPUT:

```
--- Test Cases for Fibonacci Search ---
List: [10, 20, 30, 40, 50, 60, 70, 80, 90], Target: 50, Result: 4 (Expected: 4)
List: [1, 5, 8, 12, 16, 20], Target: 7, Result: -1 (Expected: -1)
List: [5, 10, 15, 20, 25], Target: 5, Result: 0 (Expected: 0)
List: [100, 200, 300, 400, 500], Target: 500, Result: 4 (Expected: 4)
List: [], Target: 10, Result: -1 (Expected: -1)
List: [42], Target: 42, Result: 0 (Expected: 0)
List: [42], Target: 99, Result: -1 (Expected: -1)
All Fibonacci Search test cases passed!
```